

WIA 1002 DATA STRUCTURE

SEM 2, SESSION 2025/2026

Dr. Zainab Malik

zainabmalik@um.edu.my

K1 (OCC 1 – 6)

Dr. Uzair Ishtiaq

uzair@um.edu.my

K2 (OCC 7 – 12)

Ts. Dr. Siti Nurliana Binti Jamalai@Jamali

siti.nurliana@um.edu.my

K3 (OCC 13 – 15)

Home of the Bright, Land of the Brave

Di Sini Bermulanya Pintar, Tanah Tumpahnya Berani

www.um.edu.my



Recursion

- Concept
- Fibonacci Numbers
- Recursion vs Iteration
- Problem Solving

CONCEPT

Home of the Bright, Land of the Brave
Di Sini Bermulanya Pintar, Tanah Tumpahnya Berani



www.um.edu.my



[universityofmalaya](https://www.facebook.com/universityofmalaya)



[unimalaya](https://www.instagram.com/unimalaya)



[uniofmalaya](https://www.youtube.com/uniofmalaya)



UNIVERSITI
MALAYA

Recursion

- Programming technique where **a method calls itself** to fulfil its overall purpose.
- Also known as **Self-Invocation**

Characteristics of Recursion

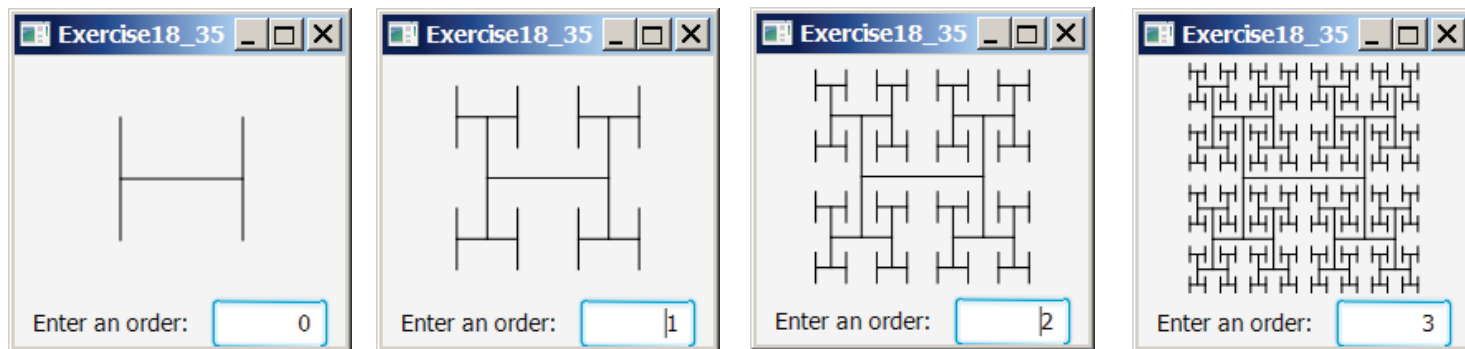
All recursive methods share two essential components:

1. **Base Case(s)**: These are the simplest scenarios where the problem can be solved directly without further recursion. They serve as the stopping condition to prevent infinite recursion.
2. **Recursive Case**: In this part, the method calls itself to solve a smaller or simpler version of the original problem. Each recursive call should move the problem closer to the base case.

Motivations

H-trees - used in a very large-scale integration (VLSI) design as a clock distribution network for routing timing signals to all parts of a chip with equal propagation delays.

How to display H-trees? A good approach is to use recursion.



Computing Factorial

$$3! = 3 * 2 * 1;$$

$$5! = 5 * 4 * 3 * 2 * 1;$$

The factorial of a number **n** can be recursively defined as follows:

$$0! = 1; \quad \text{//Base Case}$$

$$n! = n * (n - 1)!; n > 0 \quad \text{//Recursive Case}$$

Computing Factorial

How do you find $n!$ for a given n ?

- > To find $1!$ is easy, because you know that $0!$ is 1 , and $1!$ is $1 \times 0!$. Assuming that you know $(n - 1)!$, you can obtain $n!$ immediately by using $n \times (n - 1)!$. Thus, the problem of computing $n!$ is reduced to computing $(n - 1)!$. When computing $(n - 1)!$, you can apply the same idea recursively until n is reduced to 0 .

Computing Factorial

Let **factorial(n)** be the method for computing **n!**.

`factorial(0) = 1;`

`factorial(n) = n*factorial(n-1);`



ComputeFactorial

Brave

Run



LISTING 18.1 ComputeFactorial.java

```
1 import java.util.Scanner;
2
3 public class ComputeFactorial {
4     /** Main method */
5     public static void main(String[] args) {
6         // Create a Scanner
7         Scanner input = new Scanner(System.in);
8         System.out.print("Enter a nonnegative integer: ");
9         int n = input.nextInt();
10
11         // Display factorial
12         System.out.println("Factorial of " + n + " is " + factorial(n));
13     }
14
15     /** Return the factorial for the specified number */
16     public static long factorial(int n) {
17         if (n == 0) // Base case
18             return 1;
19         else
20             return n * factorial(n - 1); // Recursive call
21     }
22 }
```

base case

recursion

Enter a nonnegative integer: 4
Factorial of 4 is 24



animation

Computing Factorial

```
factorial(0) = 1;  
factorial(n) = n*factorial(n-1);
```

factorial(4)

animation

Computing Factorial

```
factorial(0) = 1;  
factorial(n) = n * factorial(n-1);
```

$\text{factorial}(4) = 4 * \text{factorial}(3)$

animation

Computing Factorial

$\text{factorial}(0) = 1;$
 $\text{factorial}(n) = n * \text{factorial}(n-1);$

$$\begin{aligned}\text{factorial}(4) &= 4 * \text{factorial}(3) \\ &= 4 * (3 * \text{factorial}(2))\end{aligned}$$

Computing Factorial

$\text{factorial}(0) = 1;$
 $\text{factorial}(n) = n * \text{factorial}(n-1);$

$$\begin{aligned}\text{factorial}(4) &= 4 * \text{factorial}(3) \\ &= 4 * (3 * \text{factorial}(2)) \\ &= 4 * (3 * (2 * \text{factorial}(1)))\end{aligned}$$

Computing Factorial

$\text{factorial}(0) = 1;$
 $\text{factorial}(n) = n * \text{factorial}(n-1);$

$\text{factorial}(4) = 4 * \text{factorial}(3)$
 $= 4 * (3 * \text{factorial}(2))$
 $= 4 * (3 * (2 * \text{factorial}(1)))$
 $= 4 * (3 * (2 * (1 * \text{factorial}(0))))$

1. temporarily suspended
until invocation complete

2. Invocation complete when it
reaches base case ($n==0$)

Computing Factorial

$\text{factorial}(0) = 1;$
 $\text{factorial}(n) = n * \text{factorial}(n-1);$

$$\begin{aligned}\text{factorial}(4) &= 4 * \text{factorial}(3) \\ &= 4 * (3 * \text{factorial}(2)) \\ &= 4 * (3 * (2 * \text{factorial}(1))) \\ &= 4 * (3 * (2 * (1 * \text{factorial}(0)))) \\ &= 4 * (3 * (2 * (1 * 1)))\end{aligned}$$

Computing Factorial

$\text{factorial}(0) = 1;$
 $\text{factorial}(n) = n * \text{factorial}(n-1);$

$$\begin{aligned}\text{factorial}(4) &= 4 * \text{factorial}(3) \\ &= 4 * (3 * \text{factorial}(2)) \\ &= 4 * (3 * (2 * \text{factorial}(1))) \\ &= 4 * (3 * (2 * (1 * \text{factorial}(0)))) \\ &= 4 * (3 * (2 * (1 * 1))) \\ &= 4 * (3 * (2 * 1))\end{aligned}$$

Computing Factorial

$\text{factorial}(0) = 1;$
 $\text{factorial}(n) = n * \text{factorial}(n-1);$

$$\begin{aligned}\text{factorial}(4) &= 4 * \text{factorial}(3) \\ &= 4 * (3 * \text{factorial}(2)) \\ &= 4 * (3 * (2 * \text{factorial}(1))) \\ &= 4 * (3 * (2 * (1 * \text{factorial}(0)))) \\ &= 4 * (3 * (2 * (1 * 1))) \\ &= 4 * (3 * (2 * 1)) \\ &= 4 * (3 * 2)\end{aligned}$$

Computing Factorial

$\text{factorial}(0) = 1;$
 $\text{factorial}(n) = n * \text{factorial}(n-1);$

$$\begin{aligned}\text{factorial}(4) &= 4 * \text{factorial}(3) \\ &= 4 * (3 * \text{factorial}(2)) \\ &= 4 * (3 * (2 * \text{factorial}(1))) \\ &= 4 * (3 * (2 * (1 * \text{factorial}(0)))) \\ &= 4 * (3 * (2 * (1 * 1))) \\ &= 4 * (3 * (2 * 1)) \\ &= 4 * (3 * 2) \\ &= 4 * 6\end{aligned}$$

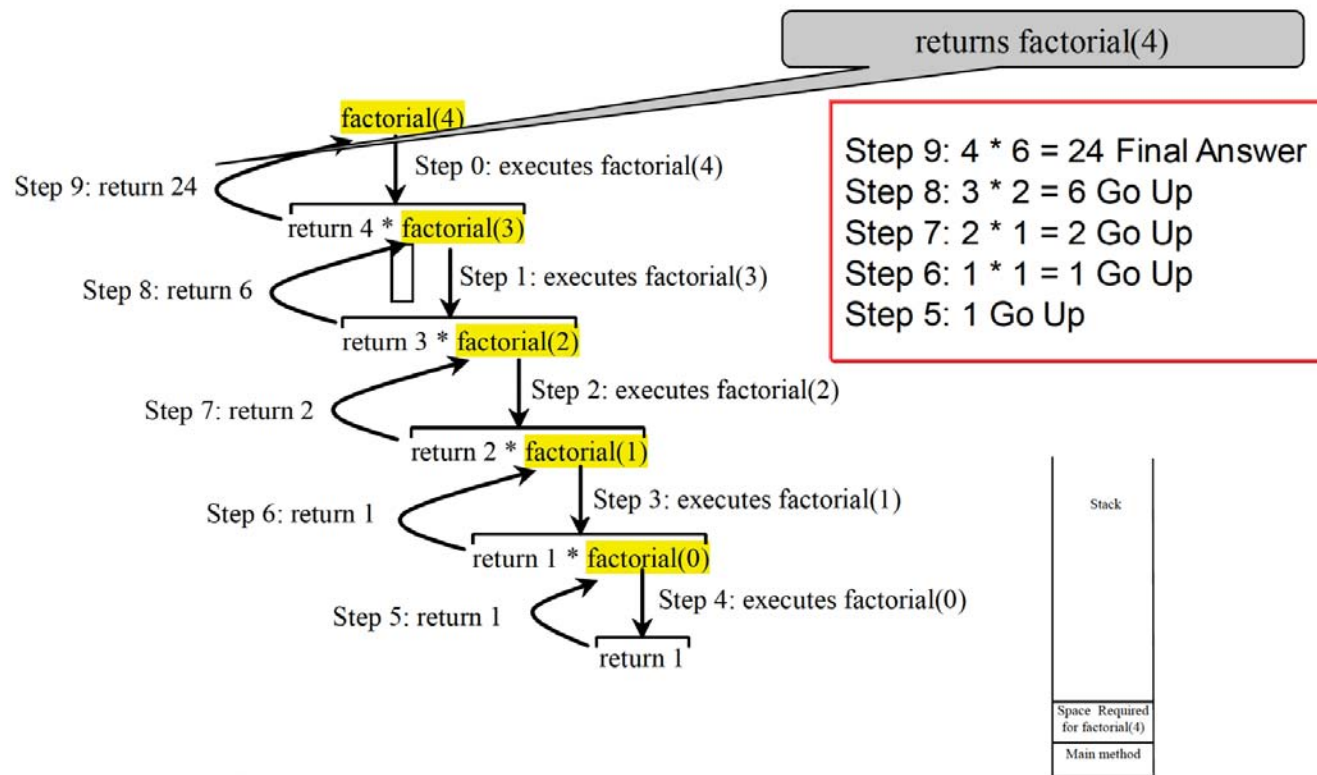
Computing Factorial

$\text{factorial}(0) = 1;$

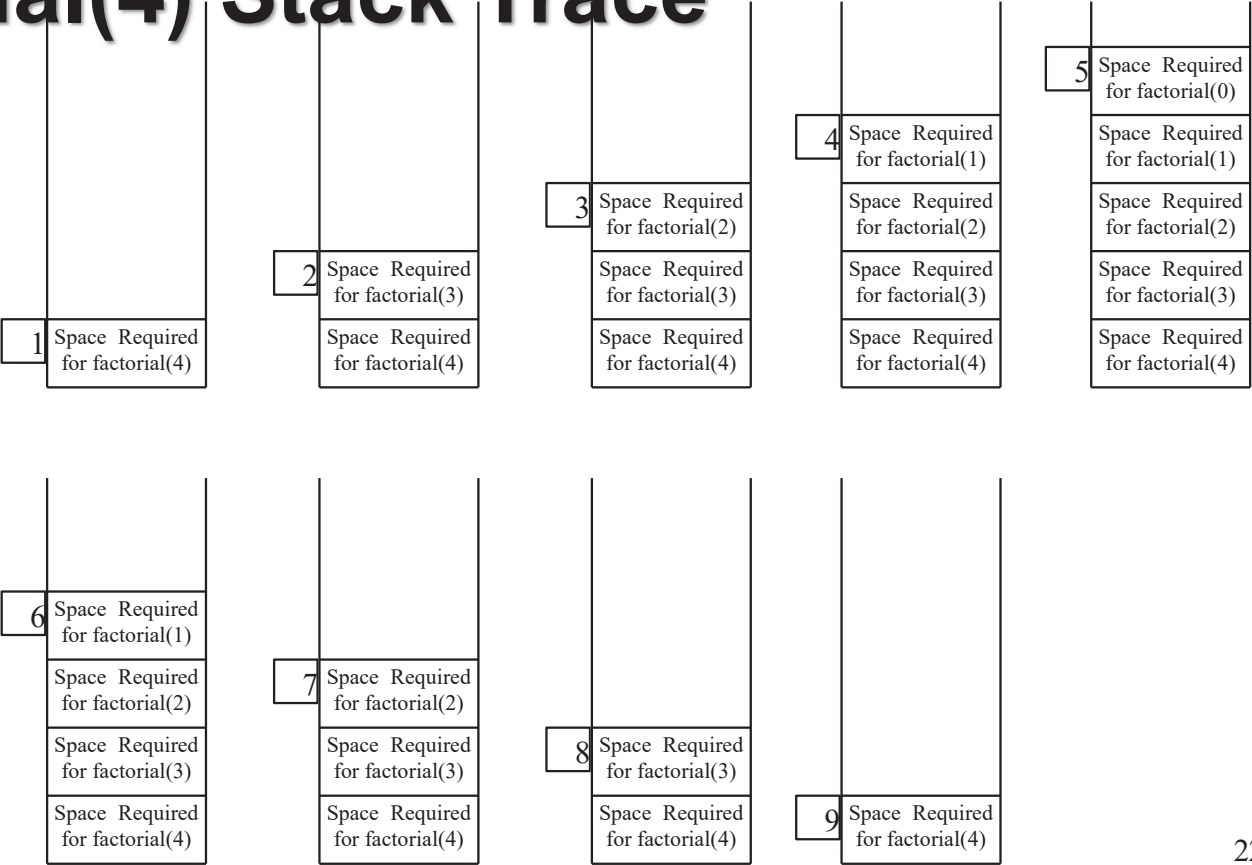
$\text{factorial}(n) = n * \text{factorial}(n-1);$

$$\begin{aligned}\text{factorial}(4) &= 4 * \text{factorial}(3) \\ &= 4 * (3 * \text{factorial}(2)) \\ &= 4 * (3 * (2 * \text{factorial}(1))) \\ &= 4 * (3 * (2 * (1 * \text{factorial}(0)))) \\ &= 4 * (3 * (2 * (1 * 1))) \\ &= 4 * (3 * (2 * 1)) \\ &= 4 * (3 * 2) \\ &= 4 * 6 \\ &= 24\end{aligned}$$

Trace Recursive factorial



factorial(4) Stack Trace



FIBONACCI NUMBERS

Home of the Bright, Land of the Brave
Di Sini Bermulanya Pintar, Tanah Tumpahnya Berani



www.um.edu.my



[universityofmalaya](https://www.facebook.com/universityofmalaya)



[unimalaya](https://www.instagram.com/unimalaya)



[uniofmalaya](https://www.youtube.com/uniofmalaya)



UNIVERSITI
MALAYA

Fibonacci Numbers

Fibonacci series: 0 1 1 2 3 5 8 13 21 34 55 89...

indices: 0 1 2 3 4 5 6 7 8 9 10 11

The Fibonacci series begins with **0** and **1**, and each subsequent number is the sum of the preceding two. The series can be recursively defined as:

$\text{fib}(0) = 0$; **//Base case**

$\text{fib}(1) = 1$; **//Base case**

$\text{fib}(\text{index}) = \text{fib}(\text{index} - 1) + \text{fib}(\text{index} - 2)$; $\text{index} \geq 2$ **//Recursive Case**

Fibonacci Numbers

Fibonacci series: 0 1 1 2 3 5 8 13 21 34 55 89...

indices: 0 1 2 3 4 5 6 7 8 9 10 11

$$\begin{aligned}\text{fib}(3) &= \text{fib}(2) + \text{fib}(1) \\ &= (\text{fib}(1) + \text{fib}(0)) + \text{fib}(1) \\ &= (1 + 0) + \text{fib}(1) \\ &= 1 + \text{fib}(1) \\ &= 1 + 1 \\ &= 2\end{aligned}$$

Fibonacci Numbers

How do you find **fib(index)** for a given **index**?

It is easy to find **fib(2)**, because you know **fib(0)** and **fib(1)**. Assuming that you know **fib(index - 2)** and **fib(index - 1)**, you can obtain **fib(index)** immediately. Thus, the problem of computing **fib(index)** is reduced to computing **fib(index - 2)** and **fib(index - 1)**. When doing so, apply the idea recursively until **index** is reduced to **0** or **1**.

The base case is **index = 0** or **index = 1**. If you call the method with **index = 0** or **index = 1**, it immediately returns the result. If you call the method with **index >= 2**, it divides the problem into two subproblems for computing **fib(index - 1)** and **fib(index - 2)** using recursive calls.

LISTING 18.2 ComputeFibonacci.java

```
1  import java.util.Scanner;
2
3  public class ComputeFibonacci {
4      /** Main method */
5      public static void main(String[] args) {
6          // Create a Scanner
7          Scanner input = new Scanner(System.in);
8          System.out.print("Enter an index for a Fibonacci number: ");
9          int index = input.nextInt();
10
11          // Find and display the Fibonacci number
12          System.out.println("The Fibonacci number at index "
13              + index + " is " + fib(index));
14      }
15
16      /** The method for finding the Fibonacci number */
17      public static long fib(long index) {
18          if (index == 0) // Base case
19              return 0;
20          else if (index == 1) // Base case
21              return 1;
22          else // Reduction and recursive calls
23              return fib(index - 1) + fib(index - 2);
24      }
25 }
```

base case

recursion

Enter an index for a Fibonacci number: 1
The Fibonacci number at index 1 is 1



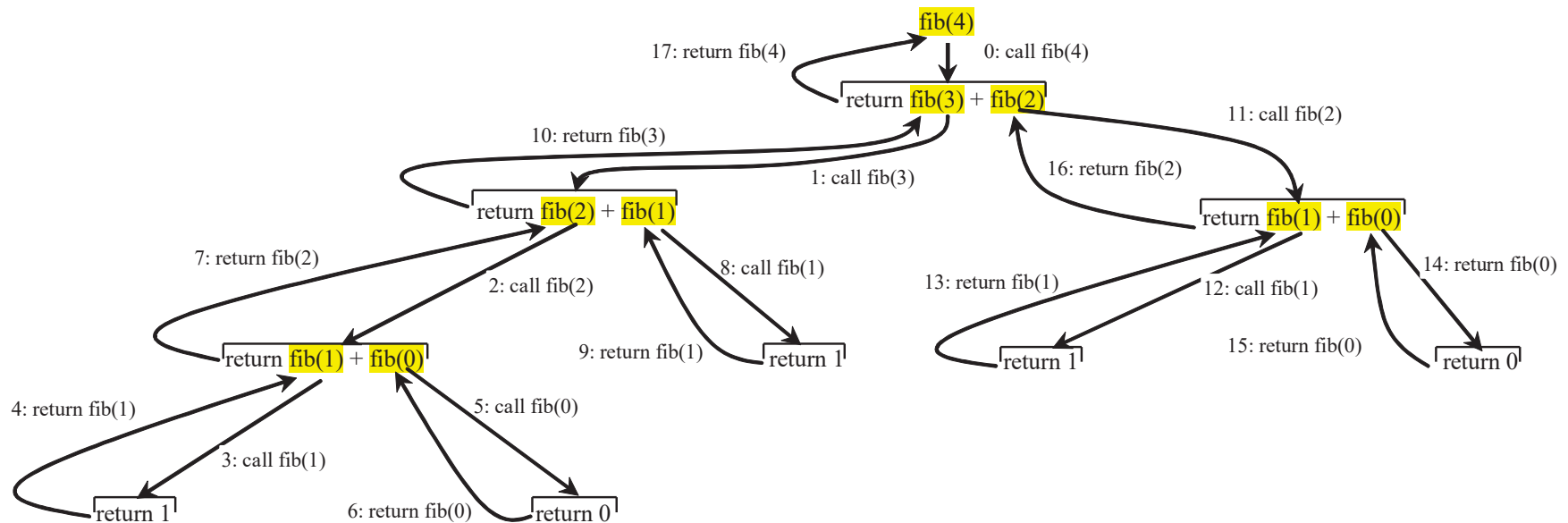
Enter an index for a Fibonacci number: 6
The Fibonacci number at index 6 is 8



Enter an index for a Fibonacci number: 7
The Fibonacci number at index 7 is 13



Fibonnaci Numbers, cont.



RECAP

Home of the Bright, Land of the Brave
Di Sini Bermulanya Pintar, Tanah Tumpahnya Berani



www.um.edu.my



[universityofmalaya](https://www.facebook.com/universityofmalaya)



[unimalaya](https://www.instagram.com/unimalaya)



[uniofmalaya](https://www.youtube.com/uniofmalaya)



UNIVERSITI
MALAYA

Characteristics of Recursion

All recursive methods share two essential components:

1. **Base Case(s)**: These are the simplest scenarios where the problem can be solved directly without further recursion. They serve as the stopping condition to prevent infinite recursion.
2. **Recursive Case**: In this part, the method calls itself to solve a smaller or simpler version of the original problem. Each recursive call should move the problem closer to the base case.

What happens if a recursive method never reaches a base case?

- Infinite recursion - occurs if recursion does not reduce the problem in a manner that allows it to eventually converge into the base case
- The stack will never stop growing.
- But OS limits the stack to a particular height, so that no program eats up too much memory.
- If a program's stack exceeds this size, the computer initiates an exception (**StackOverflowError**), which typically would crash the program.

What is the output? What is the base case?

```
public static void main(String[] args) {  
    recursion(735);  
    // System.out.println(result);  
}  
  
public static void recursion(int n) {  
    if (n > 0) {  
        System.out.print(n % 10);  
        recursion(n / 10);  
    }  
}
```

What is the output? What is the base case?

```
public static void main(String[] args) {  
    recursion(735);  
    // System.out.println(result);  
}  
public static void recursion(int n) {  
    if (n > 0) {  
        System.out.print(n % 10);  
        recursion(n / 10);  
    }  
}
```

Output : 537

Base case : $n \leq 0$

What is the output?

```
public static long factorial(int n) {  
    return n * factorial(n - 1);  
}
```

What is the output?

```
public static long factorial(int n) {  
    return n * factorial(n - 1);  
}
```

Output : The method runs infinitely and causes a StackOverflowError.

RECURSION VS ITERATION

Home of the Bright, Land of the Brave
Di Sini Bermulanya Pintar, Tanah Tumpahnya Berani



www.um.edu.my



[universityofmalaya](https://www.facebook.com/universityofmalaya)



[unimalaya](https://www.instagram.com/unimalaya)



[uniofmalaya](https://www.youtube.com/uniofmalaya)



UNIVERSITI
MALAYA

Recursion vs Iteration

- Recursion and loop are related concepts.
- Anything you can do with a loop, you can do with recursion, and vice versa.
- Sometimes recursion is simpler to write, and sometimes loop is, but in principle they are interchangeable.

Recursion vs Iteration

Implementing factorial using a loop:

```
public static long factorialLoop(int n) {  
    long result = 1;  
    while (n>0) {  
        result *= n;  
        n--;  
    }  
    return result;  
}
```

Recursion vs Iteration

Recursion

- » Terminate when a base case is reached
- » Each recursive call requires extra space on the stack frame (memory)
- » If we get infinite recursion, it may result in stack overflow

Iteration

- » Terminates when a condition is proven to be false
- » Each iteration does not require any extra space
- » An infinite loop could loop forever since there is no extra memory being created

Recursion

- An alternative form of program control.
- Repetition without a loop.
- Substantial overhead –
 - The system must assign space for all of the method's local variables and parameters each time a method is called.
 - Consume considerable memory and requires extra time to manage the additional space.
- However, it is good for solving the problems that are inherently recursive.

PROBLEM SOLVING

Home of the Bright, Land of the Brave
Di Sini Bermulanya Pintar, Tanah Tumpahnya Berani



www.um.edu.my



[universityofmalaya](https://www.facebook.com/universityofmalaya)



[unimalaya](https://www.instagram.com/unimalaya)



[uniofmalaya](https://www.youtube.com/uniofmalaya)



UNIVERSITI
MALAYA

Problem Solving Using Recursion - Think Recursively

- Example:
 - a simple problem of printing a message for n times.
 - break the problem into two subproblems:
 - one problem is to print the message one time
 - the other problem is to print the message for n-1 times. The 2nd problem is the same as the original problem with a smaller size.
 - the base case for the problem is n==0.

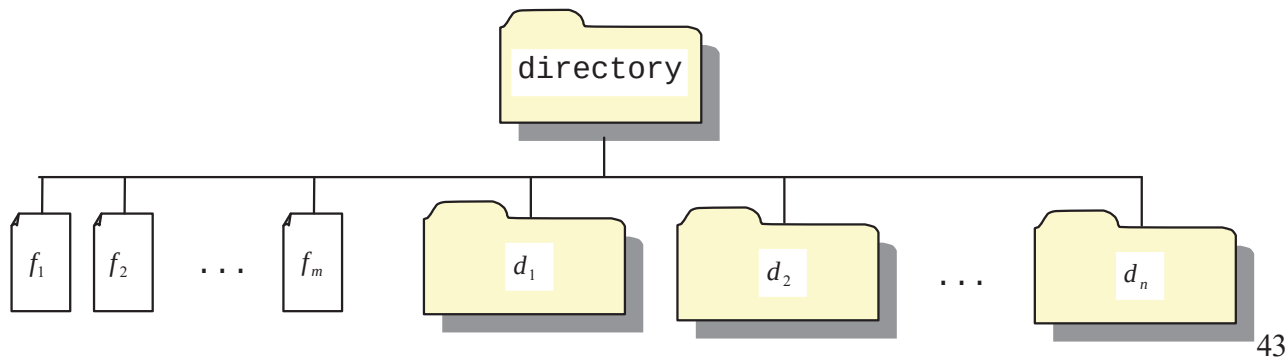
```
public static void nPrintln(String message, int times) {  
    if (times >= 1) {  
        System.out.println(message);  
        nPrintln(message, times - 1);  
    } // The base case is times == 0  
}
```

nPrintln("Welcome", 5);

42

Directory Size

- A problem that is difficult to solve without using recursion.
- The size of a directory is the sum of the sizes of all files in the directory.
- A directory may contain subdirectories.

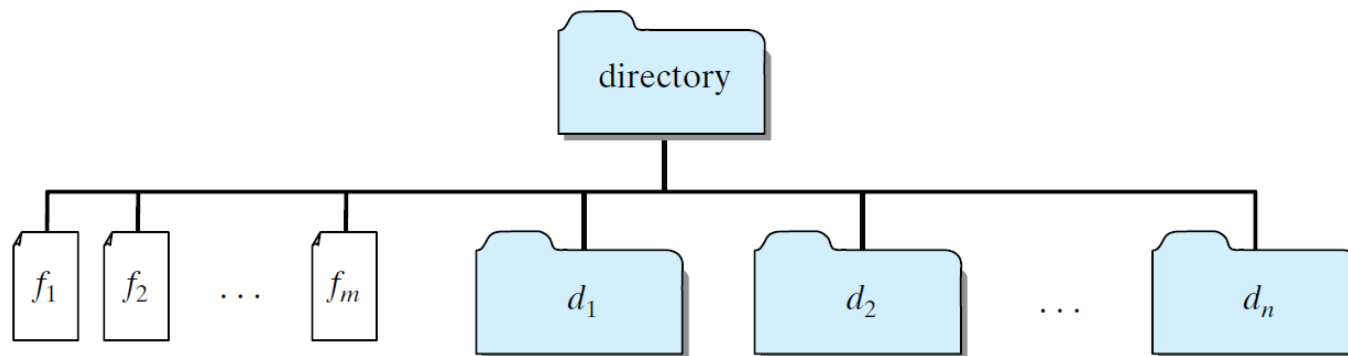


43

Directory Size

The size of the directory can be defined recursively as follows:

$$\text{size}(d) = \text{size}(f_1) + \text{size}(f_2) + \dots + \text{size}(f_m) + \text{size}(d_1) + \text{size}(d_2) + \dots + \text{size}(d_n)$$



LISTING 18.7 DirectorySize.java

```
1 import java.io.File;
2 import java.util.Scanner;
3
4 public class DirectorySize {
5     public static void main(String[] args) {
6         // Prompt the user to enter a directory or a file
7         System.out.print("Enter a directory or a file: ");
8         Scanner input = new Scanner(System.in);
9         String directory = input.nextLine();
10
11         // Display the size
12         System.out.println(getSize(new File(directory)) + " bytes");
13     }
14
15     public static long getSize(File file) {
16         long size = 0; // Store the total size of all files
17
18         if (file.isDirectory()) {
19             File[] files = file.listFiles(); // All files and subdirectories
20             for (int i = 0; files != null && i < files.length; i++) {
21                 size += getSize(files[i]); // Recursive call
22             }
23         }
24         else { // Base case
25             size += file.length();
26         }
27
28         return size;
29     }
30 }
```

Enter a directory or a file: c:\book ↵ Enter
48619631 bytes

Enter a directory or a file: c:\book\Welcome.java ↵ Enter
172 bytes

References

Chapter 18 Recursion, Liang, Introduction to Java Programming, 10th Edition, Global Edition, Pearson, 2015

Q&A

Home of the Bright, Land of the Brave
Di Sini Bermulanya Pintar, Tanah Tumpahnya Berani



www.um.edu.my



[universityofmalaya](https://www.facebook.com/universityofmalaya)



[unimalaya](https://www.instagram.com/unimalaya)



[uniofmalaya](https://www.youtube.com/uniofmalaya)



UNIVERSITI
MALAYA